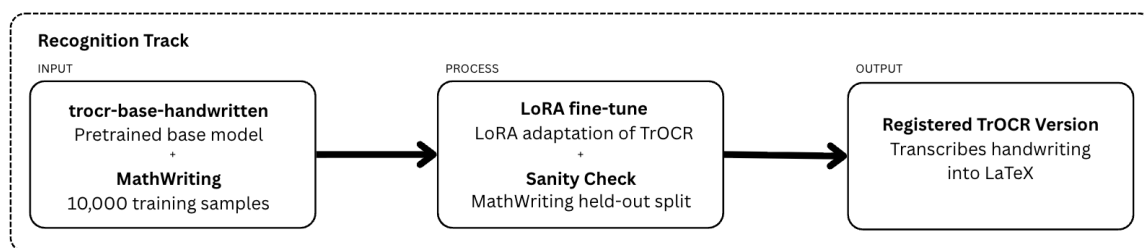


processes, and produce the registered TrOCR version as its output, as illustrated in Figure 1. The base model referenced here is the TrOCR model that pairs a pretrained ViT encoder with a pretrained text Transformer decoder (Li et al., 2023).

Figure 1

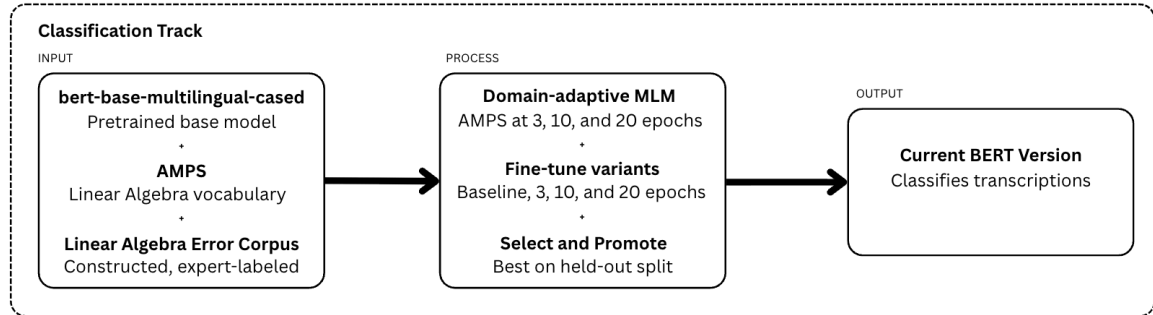
Recognition Track IPO Model



The second layer will address the semantic interpretation of that transcribed text, classifying the nature of any errors present according to a theoretically grounded taxonomy derived from established cognitive error frameworks and operationalized for the specific content of USC CS 3101N and MAT 1103, which share the same Linear Algebra content. This layer will take the base model, the AMPS Linear Algebra subset for domain-adaptive pre-training, and the Linear Algebra Error Corpus for fine-tuning as inputs, perform continued masked-language-modeling pre-training at three checkpoints, epochs 3, 10, and 20, followed by supervised fine-tuning of the non-domain-adapted baseline and the domain-adapted models, then evaluation of all four models on the held-out portion of the error corpus and selection of the best-performing variant, which is then promoted and registered as the current BERT version, as its output, as illustrated in Figure 2. The base model in this layer is BERT, a bidirectional Transformer encoder pretrained on large text corpora and designed to be fine-tuned for downstream classification tasks (Devlin et al., 2019).

Figure 2

Classification Track IPO Model

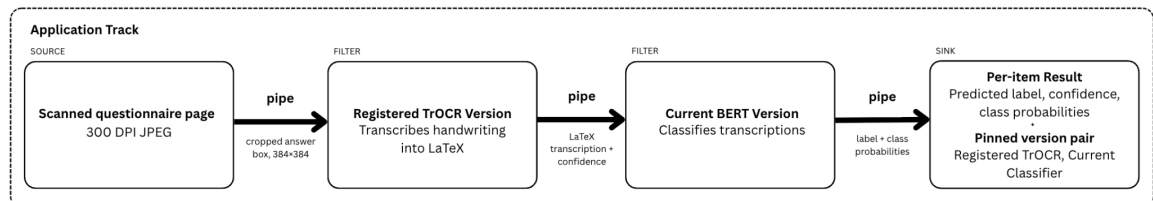


The pipeline will follow a training-then-deployment architecture. In the training phase, both layers will be fine-tuned on external, publicly available datasets before being applied to the collected student data, which will be reserved for validation and withheld from training for the duration of the validation run. This design is motivated by the absence of any publicly available dataset of annotated Linear Algebra student errors at the university level, making the local USC dataset a novel research contribution in its own right.

Under the Pipe and Filter architectural model (Shaw & Garlan, 1996), the application will process input data sequentially through interconnected filters, with each filter performing a distinct operation and forwarding its output to the next stage until the per-item result is generated, comprising the predicted label, its confidence, the class probabilities, and the pinned version pair the run executed against. Figure 3 illustrates this model.

Figure 3

Application Track Pipe and Filter Model

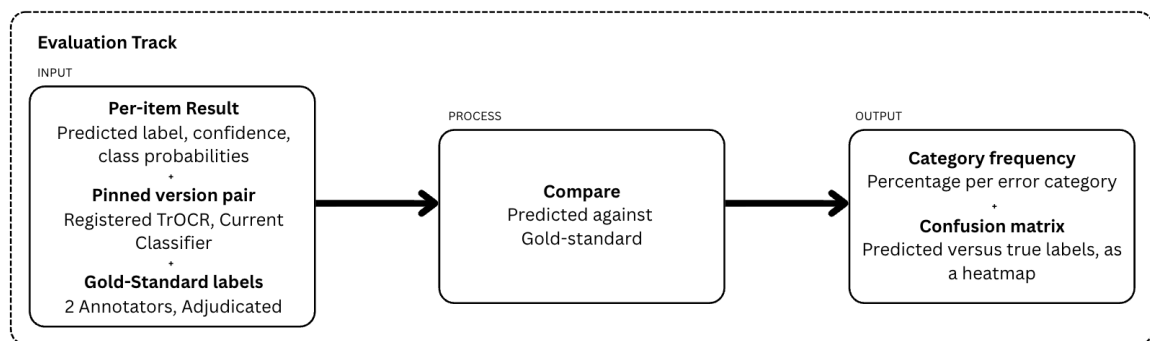


Note. Both model versions are pinned for the life of the run.

The evaluation will compare the per-item result against the gold-standard label, recording the pinned version pair alongside it so that every reported metric is attributable to an identified model state. The comparison will produce the category frequency, reporting the percentage of solutions assigned to each of the three error categories across the validation slice, and the confusion matrix, which visualizes the distribution of predicted versus true labels as a heatmap to reveal which categories the pipeline distinguishes reliably and which it tends to confuse. Figure 4 illustrates this evaluation track.

Figure 4

Evaluation Track IPO Model



Both the OCR and error-classification layers must be fully trained and validated before the system is used in practice. Once validated, each model will be registered as a numbered version and its weights will be locked for the duration of any run that uses it. Freezing is therefore a property of a run rather than a permanent property of the system. Every pipeline run will pin one OCR version and one BERT version, and no weight update will occur while that run is open, so the outputs of a run will always be attributable to a single identified model state.

The system will also support producing later versions from the labels it accumulates. Adjudicated gold-standard labels are already stored per item, which means the record needed for continued learning is generated as a by-product of normal operation. A retraining cycle will draw on that store, produce

Figure 5
PROVA Conceptual Framework

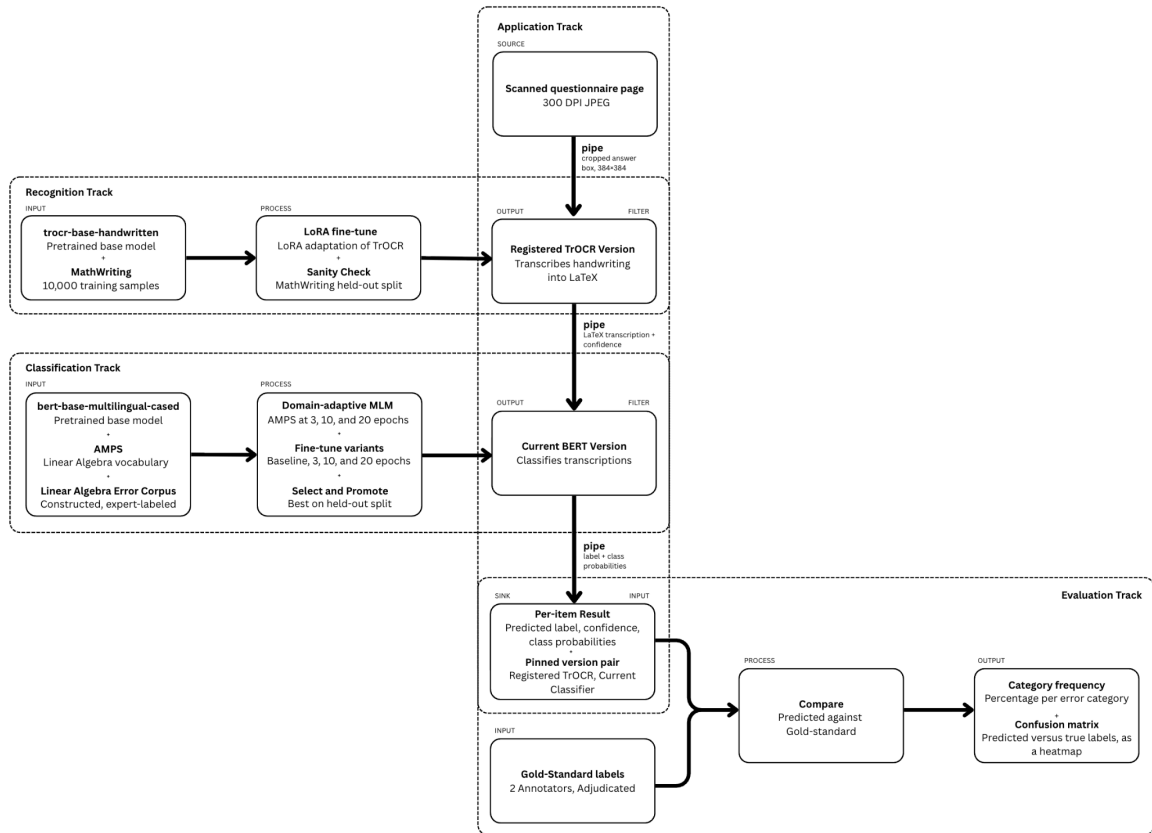
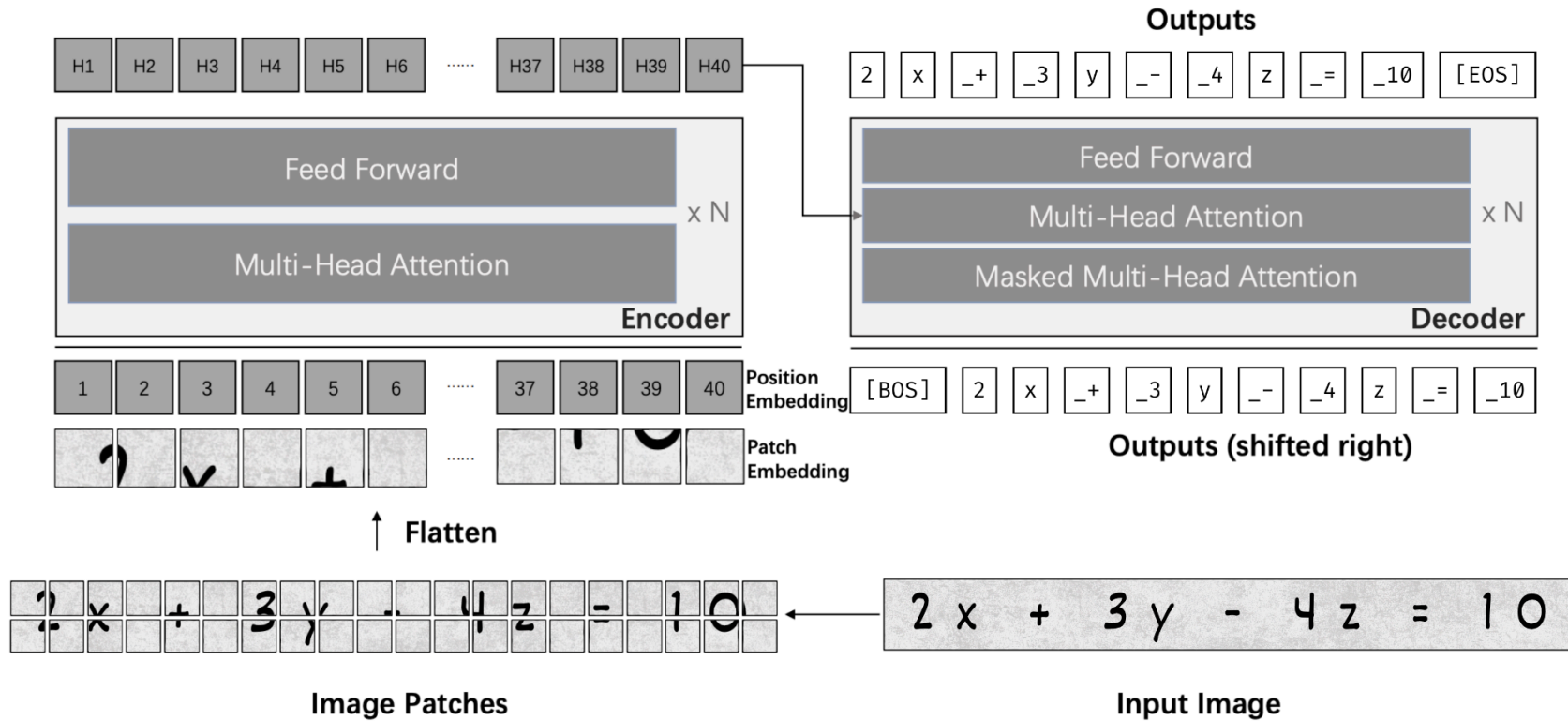


Figure 6

Recognition path of the recognition layer



Note. Adapted from "TrOCR: Transformer-Based Optical Character Recognition with Pre-trained Models" (p. 2), by M. Li et al., 2023.

Figure 7
Recognition algorithm

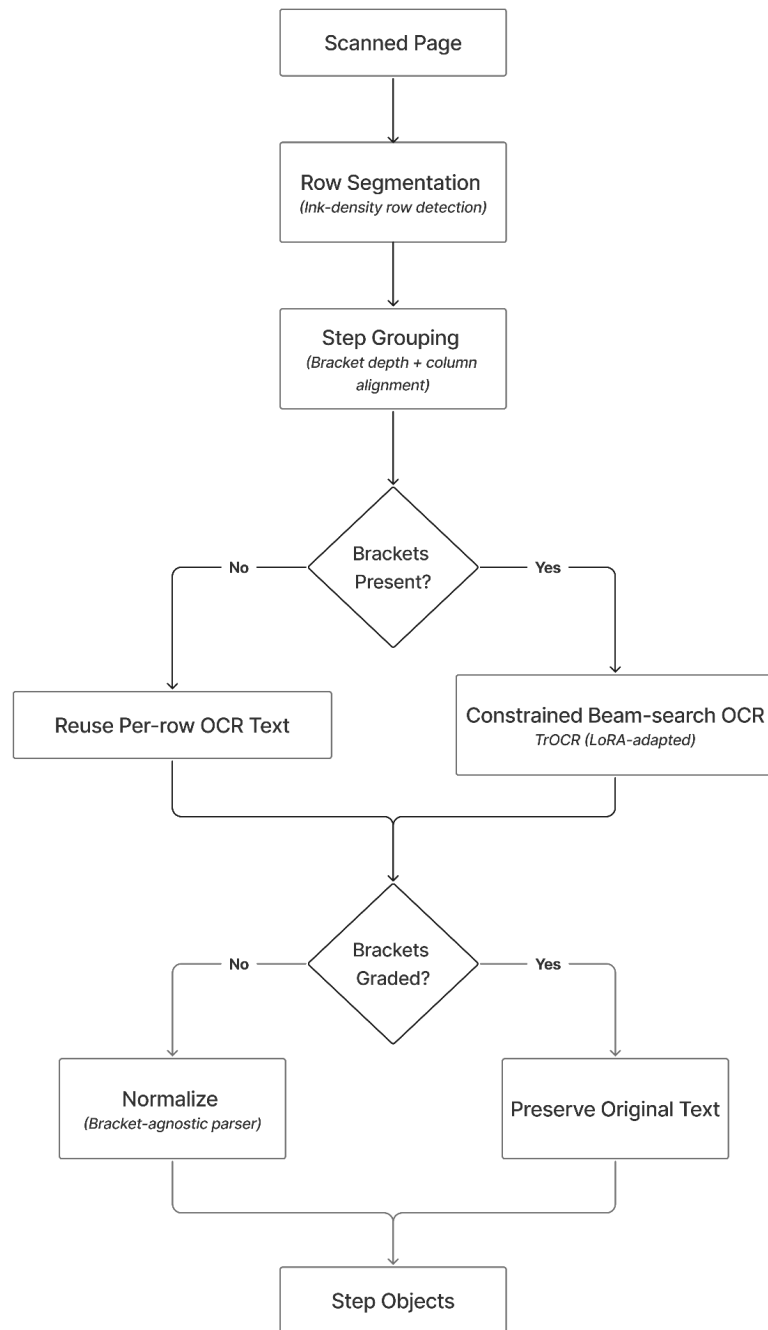
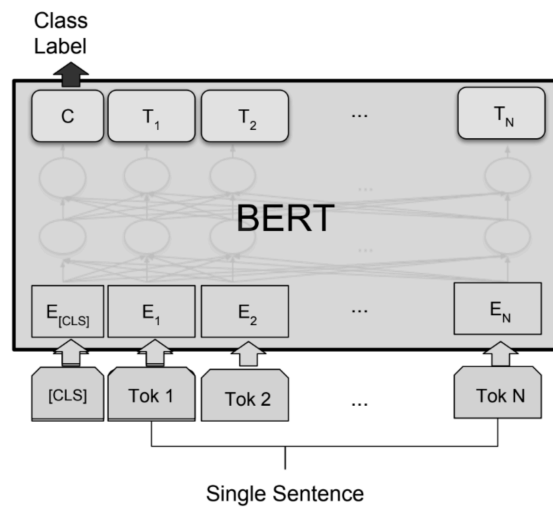


Figure 8

Classification path of the classification layer



Note. Adapted from "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding" (p. 4185), by J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, 2019, Proceedings of NAACL-HLT 2019. CC BY 4.0.

Figure 9 presents the runtime classification algorithm layered on top of the BERT classification path in Figure 8. Each Step Object is first checked for an empty solution box. Where the solution box is empty and only an answer is present, the item is tagged as having no solution and classified from the answer box alone; where a solution is present, the classifier runs on the full solution and answer together. Both cases proceed through the fine-tuned BERT model, producing a predicted category and a confidence score. Automated predictions are checked against the 0.65 confidence threshold: items at or above the threshold are recorded as automated labels, while items below the threshold, together with every no-solution item, are routed to manual classification by an expert annotator. The manual classification is then compared against the automated prediction; where the two agree, the manual label stands as the classified item, and where they disagree, the

item is routed to the adjudication protocol defined in Appendix I, Section 4.6, whose resolved label becomes the classified item.

Figure 9

Classification algorithm

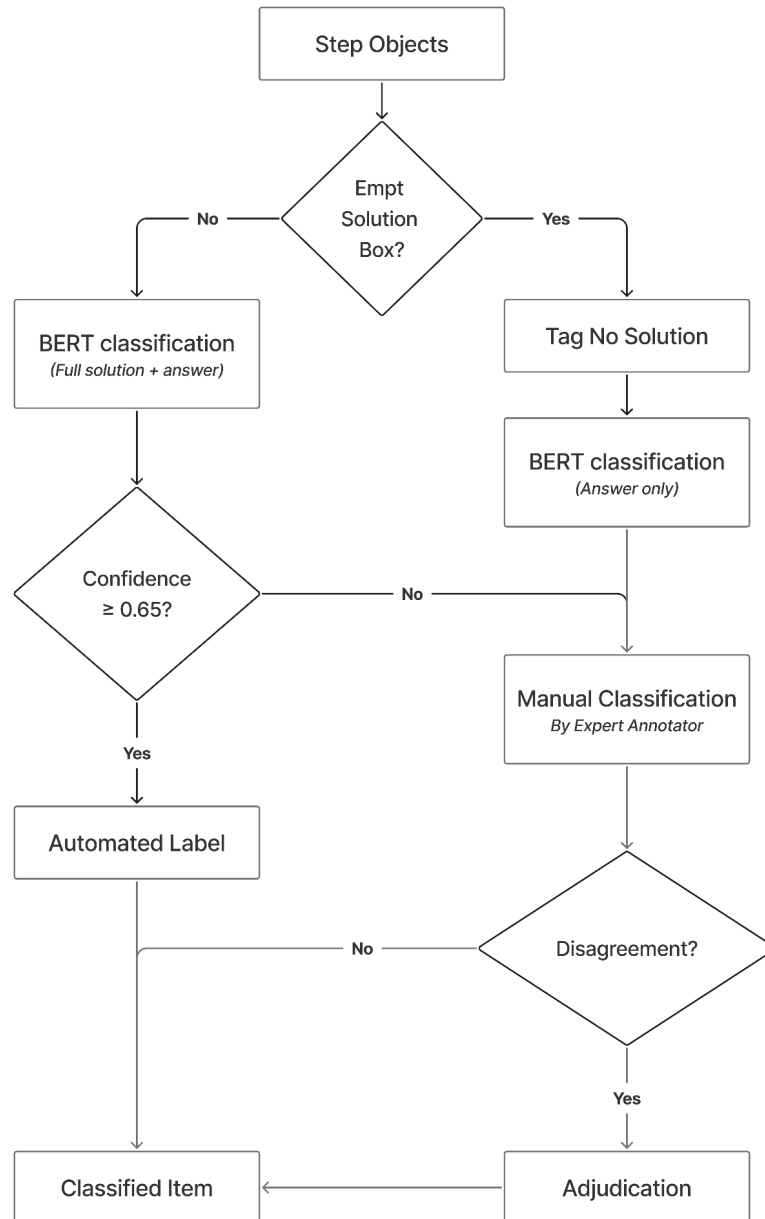


Figure 10
Evaluation flowchart

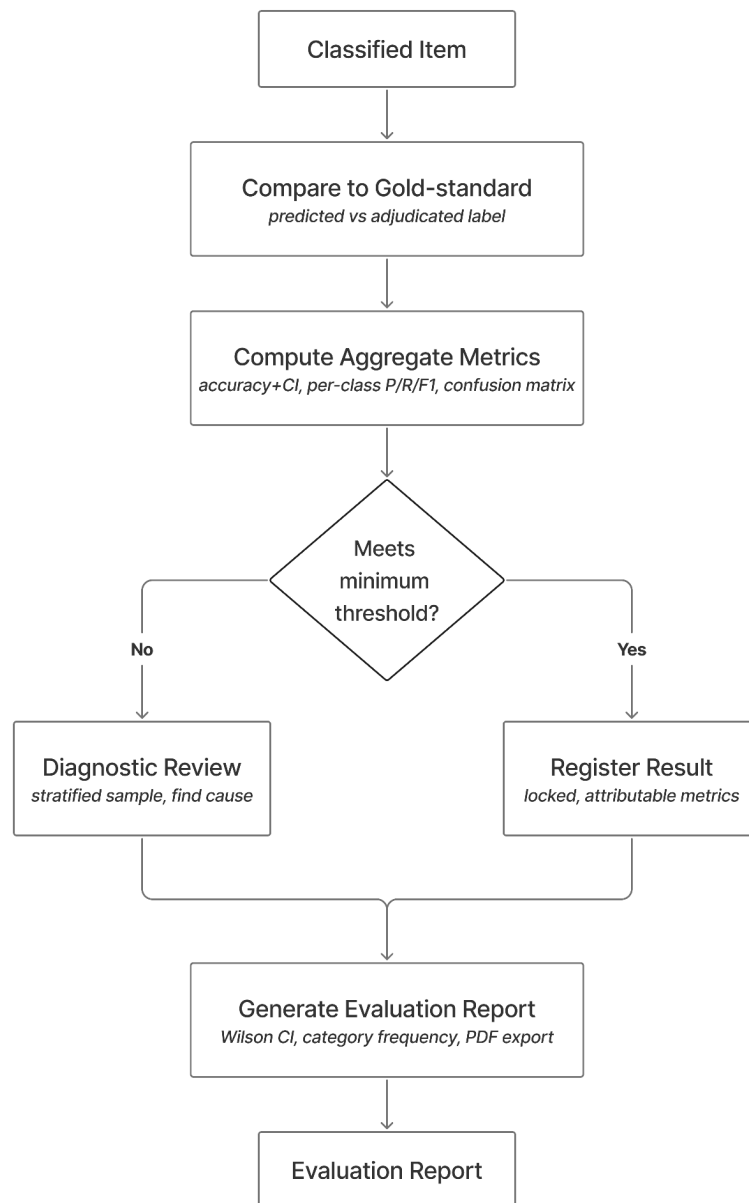
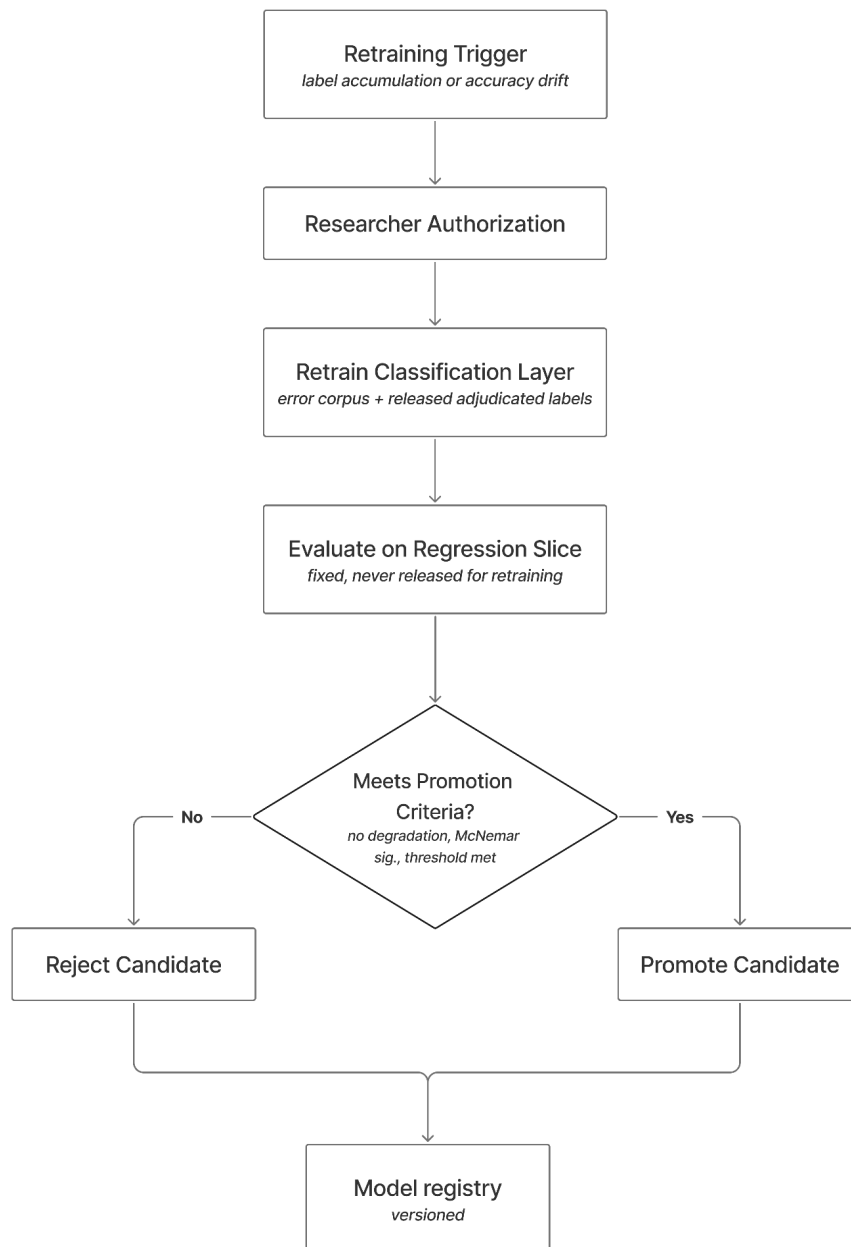


Figure 11

Retraining and promotion cycle flowchart

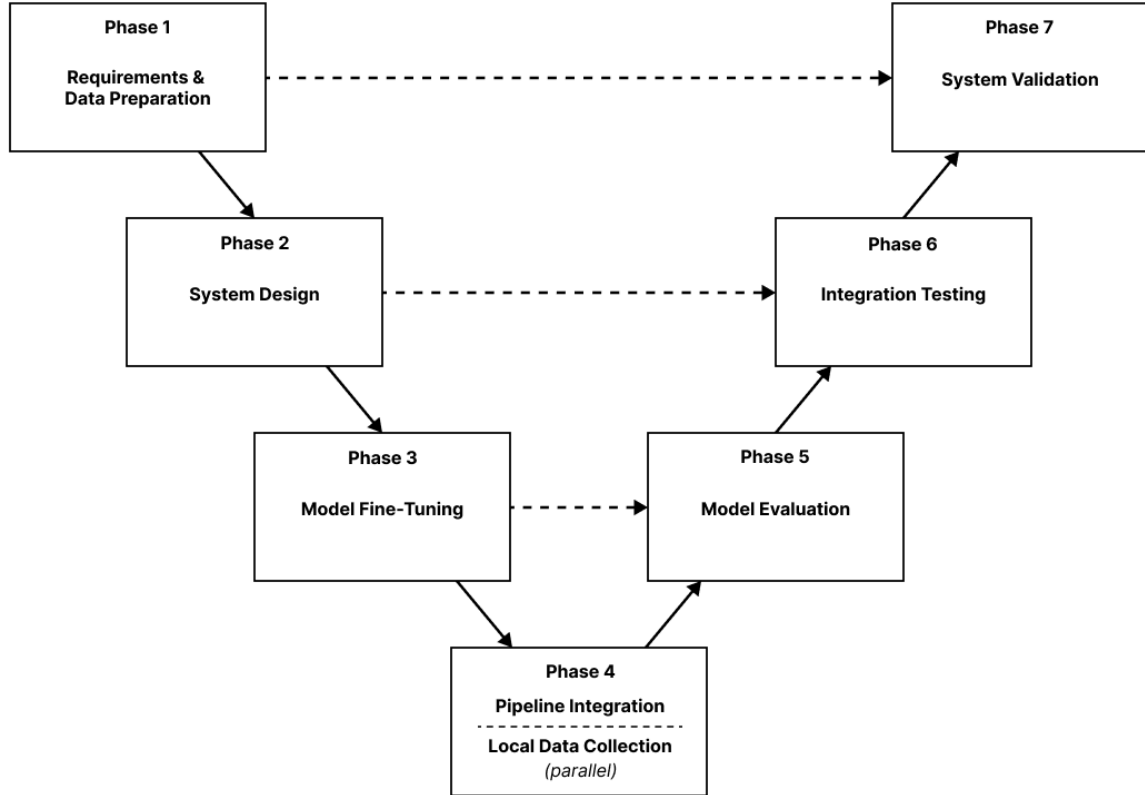


Retraining will apply to the classification layer only. The adjudicated gold-standard labels accumulated by the system are error-category labels rather than corrected transcriptions, so they supply a supervision signal for the classifier and none for the recognition layer. The recognition layer will

4.6 Development Model

Figure 12

V-Model for Cognitive Error Classification Pipeline



The development of PROVA will be guided by the Verification and Validation Model (V-Model), illustrated in Figure 12. The V-Model bends the lifecycle into two arms, pairing each decomposition and design phase on the descending arm with a corresponding integration and verification phase on the ascending arm (Rook, 1986; Forsberg & Mooz, 1991). This model is chosen because it holds each development phase accountable to a corresponding validation phase, a discipline essential for a layered AI pipeline where failure at any stage silently propagates forward.

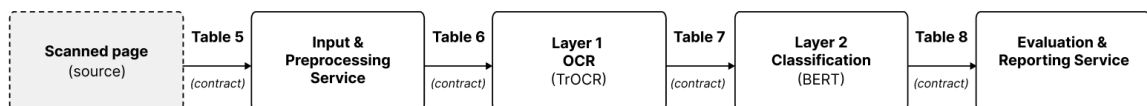
Development will begin with Phase 1, Requirements and Data Preparation, where the datasets, system requirements, and formal data contracts will be defined. Phase 2, System Design, will specify the architecture in Figure H-1. Phase 3, Model Fine-Tuning, will cover construction of the Linear Algebra

Error Corpus, domain adaptation on AMPS across the 3-, 10-, and 20-epoch conditions, LoRA fine-tuning of TrOCR, and BERT fine-tuning on the Linear Algebra Error Corpus. Phase 4, Pipeline Integration and Local Data Collection, will assemble the file-mediated pipeline while USC validation data is collected in parallel. Phase 5, Model Evaluation, will test the OCR and classifier outputs individually against their benchmarks. Phase 6, Integration Testing, will verify end-to-end pipeline behavior. Phase 7, System Validation, will confirm that the complete system satisfies all functional and non-functional requirements established in Phase 1. Phases 1 through 7 will describe a single pass through the V-Model, ending with a validated version pair. Continued learning will re-enter the model at Phase 3, Model Fine-Tuning, and will proceed forward through Phases 5 to 7 again for the candidate version, consistent with machine learning workflows in which evaluation may loop back to earlier stages rather than terminating (Amershi et al., 2019). The descending arm will not be repeated, because the requirements, architecture, and data contracts will remain unchanged across versions. Only the model artifacts will change.

4.7 Development Approaches

Figure 13

Contract-First Framework Across the PROVA Pipeline

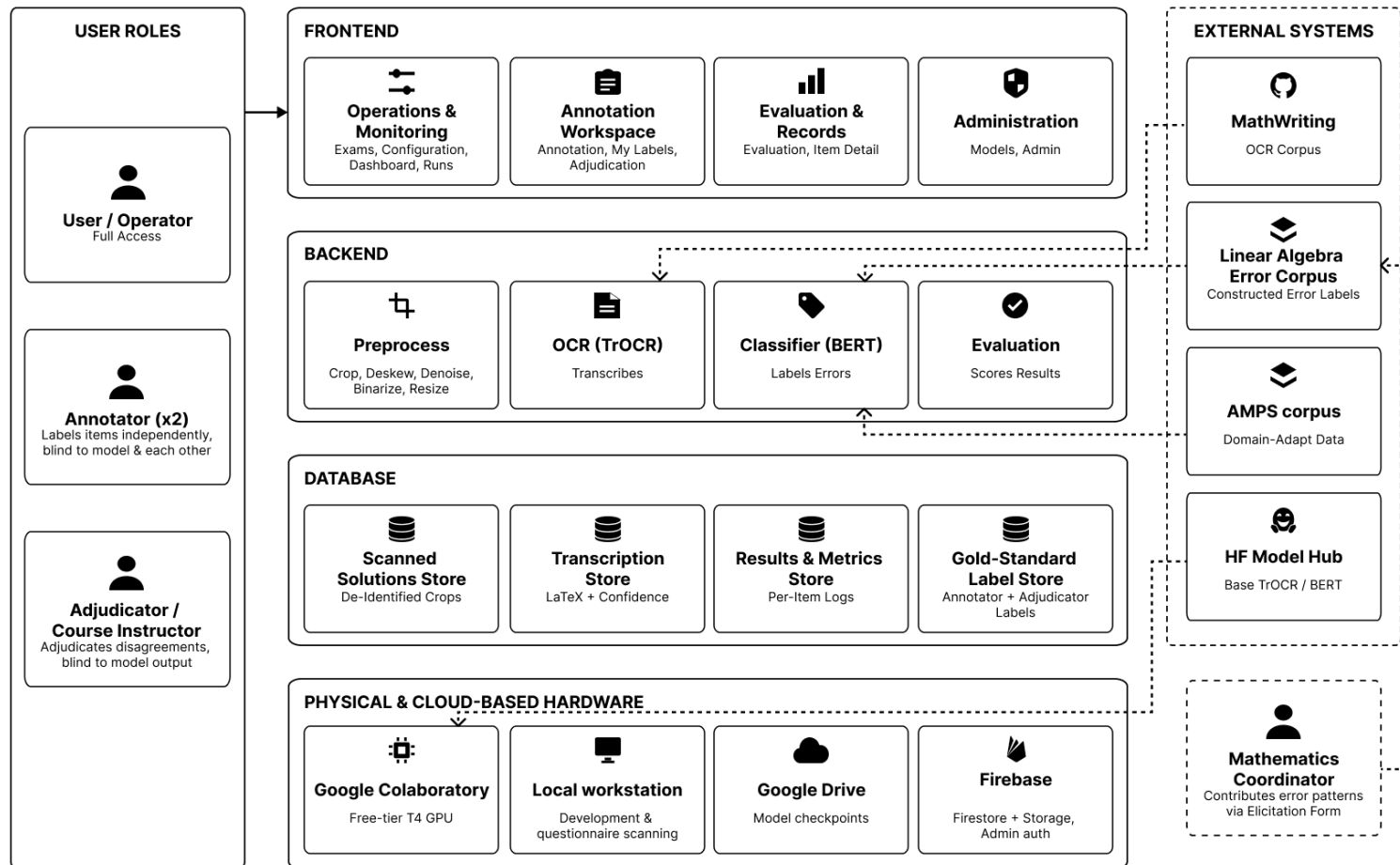


The study will adopt a contract-first development approach, in which the data contract of every component, defined as the exact format, schema, and semantics of its inputs and outputs, will be fixed before implementation begins. This approach is rooted in the Design by Contract principle of software engineering, which prescribes that interacting components be governed by formally specified mutual obligations, namely the preconditions, postconditions, and invariants fixed at each interface rather than inside any implementation (Meyer, 1992). PROVA is well suited to this approach because its core contracts

System Architecture

Figure H-1

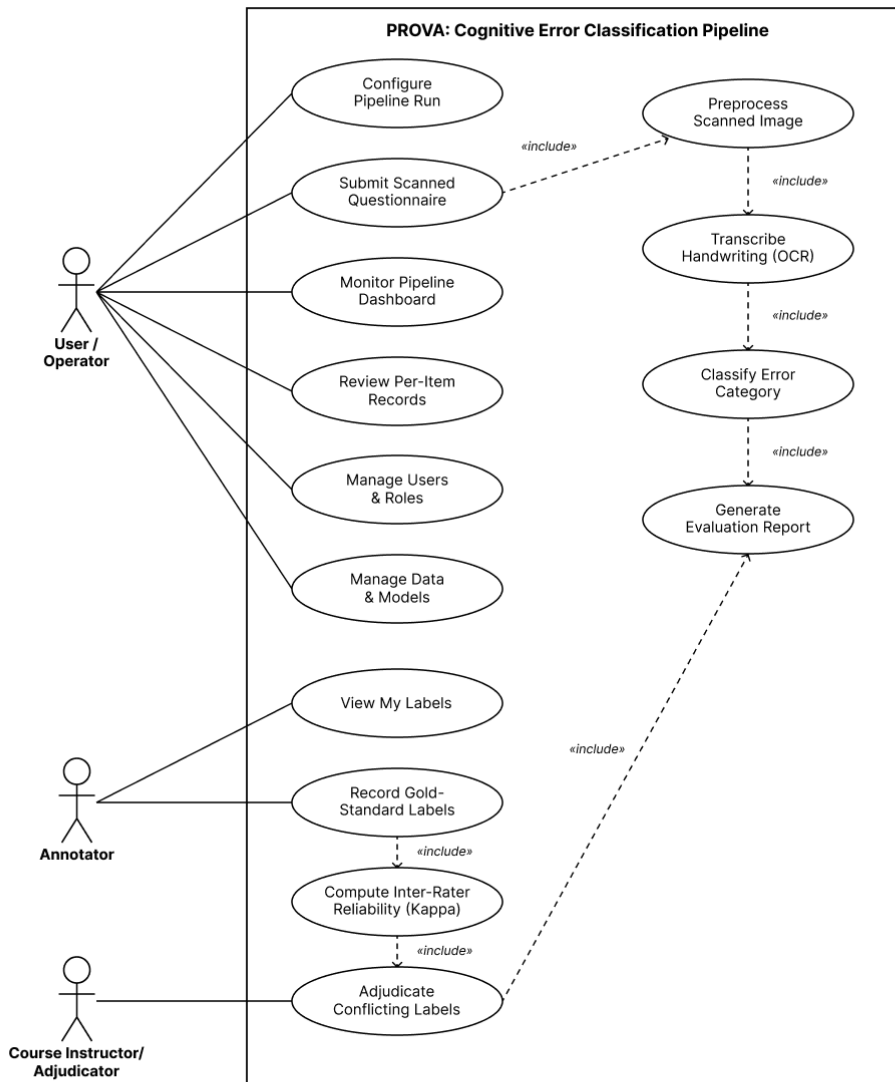
System Architecture



Use Case View

Figure H-2

Use Case Diagram



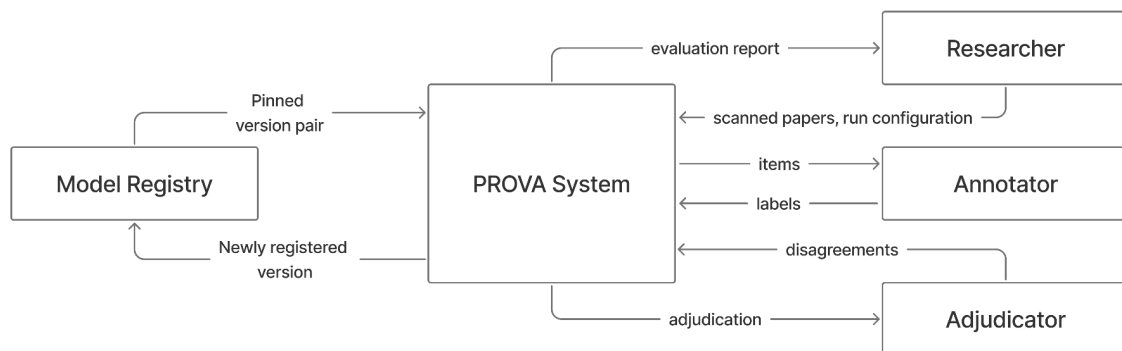
The system operates across three distinct user classes: the Researcher/Operator, the two Subject-Expert Annotators, and the Course Instructor (who serves as the Adjudicator). The Researcher/Operator has full system access, encompassing exam management, run configuration, monitoring, model version control, user administration, generating evaluation reports, authorizing retraining cycles, and promoting or rejecting candidate versions.

Promoting or rejecting a candidate version includes evaluating that candidate against the regression slice, so a promotion decision cannot be recorded without the evaluation that supports it. The Annotator is strictly limited to the Annotation and My Labels views and is entirely blind to the system's predictions and the other annotator's work. The Course Instructor is limited strictly to the Adjudication view, where they see only the solution scan, the OCR transcription, and the two conflicting annotator labels, purposefully blind to the model's output to maintain the integrity of the gold standard.

System Context

Figure H-3

Level 0 Data Flow Diagram

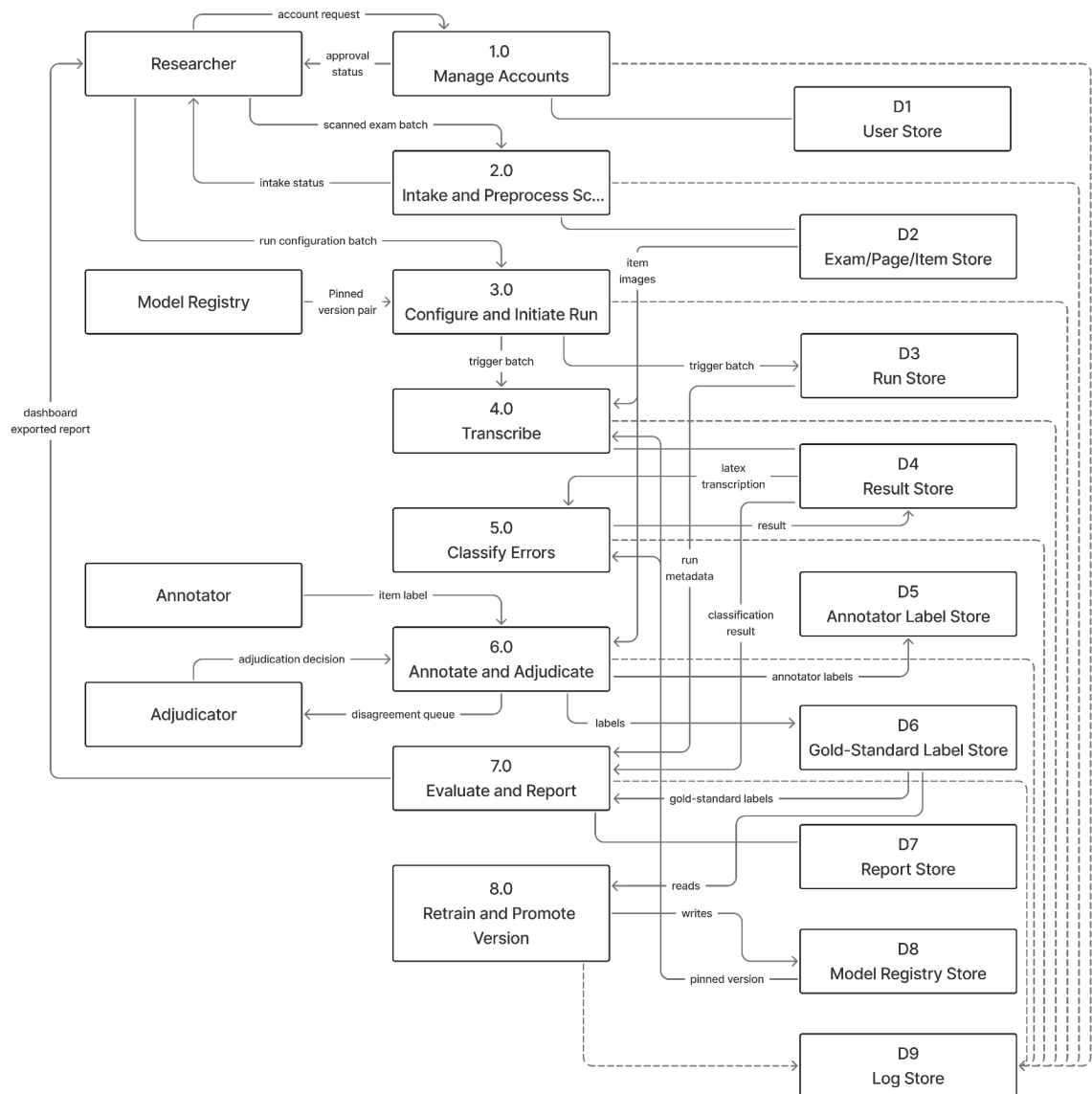


The System Context diagram models PROVA's environment and its external dependencies. The external actors interacting with the pipeline are the Researcher/Operator, the Annotator, the Adjudicator, and the Model Registry. The Model Registry supplies the pinned OCR and BERT versions a run executes against, and receives newly registered versions produced by a retraining cycle. Versions are read-only for the duration of any run that pins them, and a retraining cycle writes a new version rather than overwriting an existing one. All external interactions are channeled through gated system interfaces, ensuring that every flow crossing the system boundary is handled securely and tracked for reproducibility.

Data Flow Decomposition

Figure H-4

Level 1 Data Flow Diagram



The Level 1 Data Flow Diagram expands the system into eight functional processes and nine data stores. It follows the order of the pipeline's processing service flow. Process 1.0 handles the ingestion of scanned exam pages. Process 2.0 performs the template-based cropping and preprocessing. Process 3.0 configures and initiates a pipeline run, resolving and pinning the version pair for

Entity-Relationship Diagram



Prediction Sequence

Figure H-6

Sequence Diagram

